QuickSort vs ShellSort

PROYECTO

Kael Gonzalo Condori Fora
2026-119003
Programación avanzada
UNJBG-ESIS
Tacna-Perú

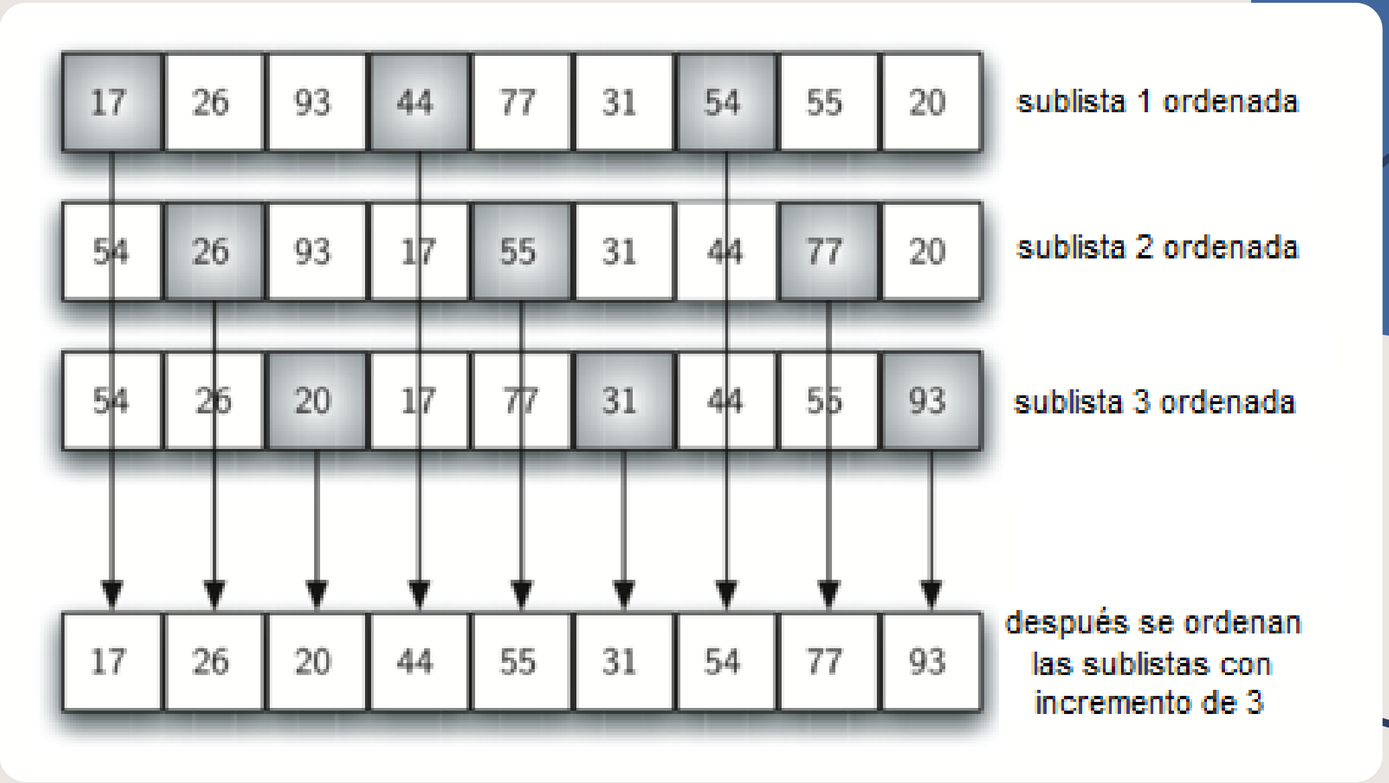
Introducción

Fundamento teórico

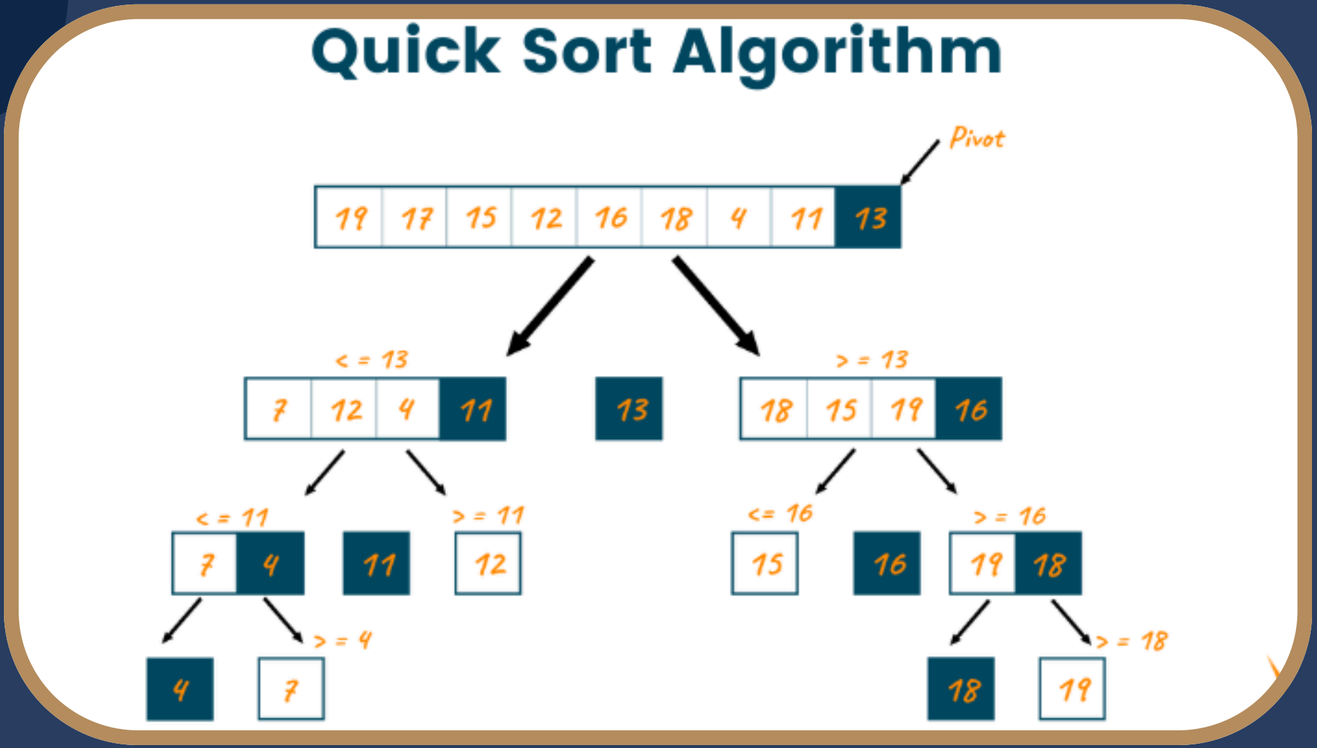
- ShellSort: Algoritmo basado en inserción con incrementos decrecientes, reduciendo desorden mediante saltos.
- QuickSort: Paradigma de Divide y Vencerás con particionamiento recursivo utilizando pivote central.
- El Problema: La discrepancia entre la complejidad teórica en el peor caso y su rendimiento real condicionado por patrones estocásticos de entrada.

ShellSort

Propuesto por Donald Shell (1959) como una evolución del ordenamiento por inserción directa. Utiliza una estrategia de subsecuencias decrementales con un salto o incremento, reduciendo el desorden general de forma progresiva.
A medida que el valor de k decrece hacia 1, las sublistas se ordenan de manera eficiente antes de la pasada final.



QuickSort



- Desarrollado por C.A.R. Hoare (1960) bajo el paradigma de diseño de algoritmos Divide y Vencerás.
- Selecciona un elemento pivote (en esta implementación, posicionado en el centro del intervalo: (k = inicio + (inicio + fin)/2) para reorganizar los elementos menores a la izquierda y los mayores a la derecha.
- Aplica llamadas recursivas independientes sobre los subarreglos resultantes.

Metodología

- Diseño experimental: Enfoque cuantitativo con un diseño factorial completo y un total de 1,500 ejecuciones independientes.
- Banco de pruebas: Implementado en C++ con memoria contigua. Se evaluaron tres tamaños de entrada (1000, 10000, 100000) y cinco patrones (Aleatorio, Ascendente, Descendente, Casi Ordenado y Duplicados) con 50 réplicas por combinación mediante mt19937.
- Métricas Registradas: Tiempo de ejecución en milisegundos (chrono), conteo de comparaciones, intercambios físicos en memoria y profundidad máxima de la pila recursiva.

Resultados

- QuickSort supera a ShellSort en velocidad bajo el patrón de entrada aleatorio 12.42 ms frente a 28.79 ms.
- ShellSort mantiene un comportamiento predecible y eficiente en escenarios con datos estructurados o parcialmente ordenados.
- QuickSort sufre una degradación severa ante el patrón de duplicados masivos, elevando su tiempo medio a 285.51 ms.
- ShellSort demuestra una alta resistencia al procesar datos duplicados, completando la tarea en 11.87 ms.

Cuadro 1: *Resumen estadístico de tiempo de ejecución (ms) para* $n = 100\,000$.

Algoritmo	Patrón	Promedio	Mediana	Desv. Est.	Mínimo	Máximo
QuickSort	Aleatorio	12.42	12.23	0.75	11.39	14.37
QuickSort	Ascendente	2.87	2.80	0.26	2.53	3.91
QuickSort	Casi_Ordenado	4.19	4.14	0.50	3.46	6.03
QuickSort	Descendente	3.31	3.22	0.34	2.95	4.96
QuickSort	Duplicados	285.51	283.02	28.88	224.53	359.91
ShellSort	Aleatorio	28.79	28.65	1.24	26.49	31.70
ShellSort	Ascendente	4.46	4.36	0.34	4.00	5.29
ShellSort	Casi_Ordenado	22.73	22.65	1.24	20.94	28.84
ShellSort	Descendente	9.24	9.14	0.53	8.39	10.54
ShellSort	Duplicados	11.87	11.65	1.08	10.50	15.08

Resultados

- QuickSort alcanza un pico de 2.13x10^8 comparaciones con datos duplicados debido a particionamientos asimétricos y desequilibrados.
- ShellSort reduce drásticamente las reubicaciones físicas, registrando 0.00 intercambios en arreglos ascendentes y 8.31x10^5 en duplicados.
- La profundidad de la pila recursiva en QuickSort se mantiene acotada dentro de log_2(n) para patrones estocásticos y ordenados.
- La presencia masiva de duplicados multiplica la profundidad de la pila en QuickSort, alcanzando un valor máximo de 6150 niveles.

Cuadro 2: *Conteo de comparaciones e intercambios promedio ($n = 100\,000$).*

Algoritmo	Patrón	Comparaciones Promedio	Intercambios Promedio
QuickSort	Aleatorio	2.71×10^6	7.49×10^5
QuickSort	Ascendente	1.50×10^6	1.00×10^5
QuickSort	Casi_Ordenado	1.60×10^6	1.61×10^5
QuickSort	Descendente	1.63×10^6	2.10×10^5
QuickSort	Duplicados	2.13×10^8	2.72×10^5
ShellSort	Aleatorio	4.34×10^6	3.66×10^6
ShellSort	Ascendente	1.50×10^6	0.00
ShellSort	Casi_Ordenado	3.42×10^6	2.54×10^6
ShellSort	Descendente	2.24×10^6	1.66×10^6
ShellSort	Duplicados	2.02×10^6	8.31×10^5

Cuadro 3: *Profundidad máxima y promedio de la pila recursiva en QuickSort para todos los tamaños y patrones.*

Tamaño (n)	Patrón	Profundidad Promedio	Profundidad Máxima
1 000	Aleatorio	20.52	25
1 000	Ascendente	9.00	9
1 000	Descendente	9.00	9
1 000	Casi_Ordenado	11.24	14
1 000	Duplicados	76.10	91
10 000	Aleatorio	30.48	38
10 000	Ascendente	13.00	13
10 000	Descendente	13.00	13
10 000	Casi_Ordenado	15.60	19
10 000	Duplicados	670.14	756
100 000	Aleatorio	41.12	49
100 000	Ascendente	17.00	17
100 000	Descendente	17.00	17
100 000	Casi_Ordenado	20.45	24
100 000	Duplicados	5420.80	6150

Discusión

- Jerarquía de Memoria Caché: Aunque ShellSort realiza más comparaciones aleatorias que QuickSort, la diferencia temporal es menor 16 ms gracias a los saltos decrecientes sobre bloques contiguos que optimizan el uso de la caché (niveles L1-L3).
- Impacto Crítico de los Duplicados: La variante de QuickSort con pivote central colapsa ante alta densidad de claves idénticas 285.51 ms y 2.13x10^8 comparaciones debido a particiones asimétricas que degradan el rendimiento hacia O(n^2).
- Resiliencia de ShellSort: Demuestra inmunidad ante patrones repetitivos 11.87 ms gracias a su esquema de preclasificación por incrementos.
- Seguridad en la Pila: Los duplicados multiplican por más de 20 veces la profundidad de la pila en QuickSort (hasta 6150 niveles), representando un riesgo de stack overflow, mientras que ShellSort opera de forma estrictamente iterativa con O(1) de memoria auxiliar.

Conclusiones

• Hegemonía de QuickSort

Confirma su superioridad temporal en distribuciones estocásticas y estructuradas sin repetición masiva, siendo hasta 2.3 veces más rápido que ShellSort para n = 100000.

• Vulnerabilidad Crítica

QuickSort con pivote central sufre una degradación severa ante duplicados masivos, requiriendo migrar a esquemas de particionamiento en tres vías o algoritmos introspectivos.

• Robustez y Predictibilidad

ShellSort destaca por su diseño iterativo e in-place, previniendo desbordamientos de pila y aprovechando eficientemente la localidad de referencia en arreglos preordenados o repetitivos.

• Criterio de Selección

La elección algorítmica en ingeniería no debe basarse únicamente en la teoría asintótica; es indispensable evaluar el hardware, los límites de la pila y la naturaleza estocástica de los datos.

Bibliografía

- Brassard, G., & Bratley, P. (1997). Fundamentos de Algoritmia. Prentice Hall.
- Ciura, M. (2001). Best increments for the average case of Shellsort. Fundamentals of Computation Theory, 2138, 106-117. https://doi.org/10.1007/3-540-44759-8_12
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introducción a los Algoritmos (3rd). Cengage Learning. https://mitpress.mit.edu/9780262033848/introduction- to-algorithms/
- Frigo, M., Leiserson, C. E., Prokop, H., & Ramachandran, S. (1999). Cache-oblivious algorithms. Proceedings of the 40th Annual Symposium on Foundations of Computer Science (FOCS), 285-297. https://doi.org/10.1109/SFCS.1999.814590
- Hoare, C. A. R. (1962). Quicksort. The Computer Journal, 5(1), 10-16. https://doi.org/10.1093/comjnl/5.1.10
- Knuth, D. E. (1998). The Art of Computer Programming, Volume 3: Sorting and Searching (2nd). Addison-Wesley. https://vivekupadhyay125.wordpress.com/wp-content/uploads/2013/08/donald-e-knuth-the-art-of-computer-programming-vol-3.pdf
- Musser, D. R. (1997). Introspective sorting and selection algorithms. Software: Practice and Experience, 27(8), 983-993. https://doi.org/10.1002/(SICI)1097- 024X(199708)27:8<983::AID-SPE117>3.0.CO;2-#
- Rodríguez, C. A., & Gómez, F. (2010). Análisis Comparativo y Empírico de Algoritmos de Ordenamiento en Estructuras Contiguas. Revista Iberoamericana de Inteligencia Artificial, 14(45), 23-35. https://dialnet.unirioja.es/descarga/articulo/4527973.pdf
- Sedgewick, R. (1978). Implementing Quicksort programs. Communications of the ACM, 21(10), 847-857. https://doi.org/10.1145/359619.359631
- Shell, D. L. (1959). A high-speed sorting procedure. Communications of the ACM, 2(7), 30-32. https://doi.org/10.1145/368370.368387
- Weiss, M. A. (2013). Estructuras de Datos y Algoritmos en C++ (4th). Pearson Educación. https://www.uoitc.edu.iq/images/documents/informatics-institute/Competitive_exam/DataStructures.pdf

Muchas

GRACIAS